Software Requirements Specifications

UniVerseX

Project Advisor

Ayesha Zaheer

Presented by:

L1F24BSCS0494 **Sania Nasir**
L1F24BSCS0340 **Seerat Fatima**
L1F24BSCS0506 **Sharjeel Ahmad**

Table of Contents

INTRODUCTION AND BACKGROUND

UniVerseX is a mobile application designed to create a social platform for university students to communicate, share knowledge, and conduct peer-to-peer transactions. Currently, students' discussions scatter across WhatsApp, Facebook, and Instagram, making important information difficult to retrieve and maintain continuity. Our platform consolidates academic discussions, social interaction, and student commerce into a single, structured environment accessible on both iOS and Android devices.

1.1. Problem Statement

University students face fragmented communication channels where academic discussions, social posts, and marketplace activity are scattered across multiple unstructured platforms. This fragmentation results in:

- Loss of important academic information in group chats
- Difficulty finding answers to repeated questions
- Unsafe, unmoderated student commerce
- No dedicated space for campus-specific discussions

UniVerseX solves this by providing a unified platform where students can:

- Post and discuss course-related topics in moderated communities
- Share confessions and social content anonymously
- Buy, sell, and barter textbooks, notes, and campus accessories
- Access all information within a single, organized interface in various dedicated communities

1.2. Background

In universities, communication is essential for academic success and campus life. Students currently rely on general-purpose social media platforms not specifically designed for academic purposes. WhatsApp groups become cluttered, Facebook groups are public and unprofessional, and Instagram stories fade within 24 hours. There is a clear demand for a dedicated student platform that is community-driven.

The proposed solution draws inspiration from successful platforms: **Reddit, Piazza** and **Empor** remaining lightweight and prototype-focused using **MIT App Inventor**. The platform will emphasize moderation, user safety, and organized architecture to ensure a healthy discussion environment.

1.3. Scope

UniVerseX will be developed as a cross-platform mobile application (iOS/Android) with the following core features:

Authentication & Profile:

- ◆ Email/password registration and login
- ◆ User profile with username and reputation score
- ◆ Profile view showing user's posts and marketplace listings

Discussion Features:

- ◆ Create text-based posts with optional image uploads
- ◆ Post categorization (Question, Discussion, Confession)
- ◆ Moderated communities
- ◆ Comment threads under posts
- ◆ Upvote/downvote mechanism for post visibility
- ◆ Search posts by keyword, category, or community (Limited)

Marketplace Features:

- ◆ List items for sale or barter (books, notes, accessories, etc.)
- ◆ Item details with images, price, and barter options
- ◆ Contact seller through in-app messaging
- ◆ User ratings for marketplace transactions

Moderation:

- ◆ Community moderators (admins) appointed per community
- ◆ Ability to report inappropriate content
- ◆ Admin dashboard for content removal
- ◆ Spam and abuse prevention

Technical Stack:

- ◆ Frontend: MIT App Inventor (cross-platform blocks)
- ◆ Backend: Firebase (Realtime Database + Authentication)
- ◆ Image Storage: Cloudinary (free tier)
- ◆ Target: Android + iOS

1.4. Objectives

- ➢ Provide a centralized, organized platform for student communication and knowledge sharing
- ➢ Eliminate information fragmentation across multiple apps
- ➢ Create safe, moderated spaces for academic and social discussions
- ➢ Enable peer-to-peer student commerce with built-in trust mechanisms
- ➢ Improve student engagement and sense of campus community
- ➢ Demonstrate practical application of software engineering principles in a real-world project

1.5. Challenges

Real-time Synchronization: Ensuring feed updates reflect new posts and comments instantly across users

Database Complexity: Managing relationships between users, posts, comments, and marketplace listings

Image Handling: Optimizing image uploads and storage within app size constraints

User Trust: Building confidence in marketplace transactions (ratings, reviews, dispute resolution)

Performance: Managing and organizing a large number of posts and comments

MIT App Inventor Limitations: Lack of advanced features like *nested* queries and complex business logic

1.6. Learning Outcomes

Upon completion, team members will have gained:

- Understanding of *Software Requirements Specification* documentation
- Mobile app development using MIT App Inventor and visual programming
- Cloud database design and real-time synchronization with Firebase
- User authentication and security best practices
- Marketplace and community platform architecture
- Team collaboration and version control (document management)
- Understanding user interaction and *system design*
- ***Software engineering lifecycle*** from specification to prototype

1.7. Nature of End Product

UniVerseX will be delivered as a **functional prototype** demonstrating core functionality:

- Cross-Platform app
- Fully functional discussion and post features
- Working marketplace with image uploads
- Real-time Firebase backend integration
- Moderated community system
- User authentication and profile management

The prototype prioritizes functionality over polish; advanced features like performance optimization and complex analytics are **excluded**.

1.8. Completeness Criteria

The project will be considered complete when:

1. Users can register, log in, and maintain profiles
2. Posts can be created, displayed, and deleted by authors
3. Comments appear under posts in real-time
4. Upvote/downvote system functions correctly and reflects in post scoring
5. Communities can be created and moderated
6. Marketplace listings can be posted with images and descriptions
7. Search functionality returns relevant results
8. Feed displays posts sorted by newest and popularity
9. Marketplace transactions show contact options

1.9. Business Goals

- ◆ Increase student engagement within the university ecosystem
- ◆ Provide a safe alternative to unmoderated third-party platforms
- ◆ Reduce dependency on external social media for campus discourse
- ◆ Enable sustainable student commerce with built-in buyer protection
- ◆ Build a foundation for potential university adoption and expansion

1.10. Related Works

Reddit (reddit.com): Community-driven discussion platform with upvote/downvote ranking, moderated subreddits, and user reputation. UniVerseX adopts the moderated community model and post-based discussion format.

Piazza (piazza.com): Academic Q&A platform used in universities with instructor monitoring and structured answers. UniVerseX incorporates the educational focus and moderation hierarchy.

OLX / Facebook Marketplace: Peer-to-peer marketplace platforms. UniVerseX integrates marketplace functionality with trust ratings and in-app messaging.

UniVerseX differentiates itself by combining all three elements (*discussion, academic focus, and marketplace*) in one platform specifically designed for universities, rather than adapting general-purpose tools.

1.11. Document Conventions

Key Terms: User, Post, Comment, Community, Moderator, Listing, Marketplace are used consistently throughout
Terminology: "Post" refers to discussion content; "Listing" refers to marketplace items
Hierarchy: Communities contain Posts; Posts contain Comments; Marketplace is a separate feature area
Numbering: Sections follow **IEEE SRS** standard numbering format

OVERALL DESCRIPTION

2.1. Product Features

The major features of the system are listed below:

A. Discussion & Posts

- Create posts with title, body text, tags and optional images
- **Tags**: Question, Discussion, Confession, etc. to improve organization and searchability
- Upvote/downvote mechanism with score calculation
- Comment threads on each post
- Edit and delete own posts
- Pin important posts (moderators only)
- Search posts by keyword, tag, or community

B. Communities

- Create communities around topics, courses, or interests
- Join communities
- View community-specific feed
- Moderator appointment and role management
- Community rules and description
- Community moderation tools (pin, remove posts)

C. Marketplace

- List items for sale or barter with images
- Item categories (Books, Notes, Accessories, Electronics, etc.)
- Seller contact information and ratings
- In-app messaging between buyer and seller
- Transaction status tracking (Available, Sold, Exchanged)
- User reputation/ratings system

D. User & Authentication

- Email/password registration and login (**must be university mail**)
- Profile customization (username, bio, profile picture)
- View user's posts and marketplace listings
- Reputation score (based on upvotes and transaction ratings)
- Follow users (optional, for future phases)

E. Moderation & Safety

- Report inappropriate content
- Moderator dashboard per community
- Content removal and user warnings
- Spam detection and prevention
- Transaction dispute resolution tools

2.2. User Classes and Characteristics

1. Regular Students (Primary Users)

- University students aged 18 – 25
- Basic to intermediate mobile app experience
- Use the app for academic discussions, social posts, and buying/selling items
- Expect fast, intuitive interface with minimal learning curve
- May create 5 – 10 posts per week on average

2. Moderators (Secondary)

- Experienced users appointed by admins or community creators
- *Responsibility*: monitor content, remove spam, enforce community guidelines
- Higher privileges: ability to pin/remove posts, warn users
- Typically 1 – 2 per community of 100+ members

3. Administrators (System Admins)

- *Italics* indicate special role; appointed by development team
- Full platform access: view all posts, remove any content, manage moderators
- Ability to disable accounts and enforce platform-wide policies

4. Guest Users (Optional, Future Phase)

- Non-registered users who can view public posts (read-only)
- Cannot create posts, comment, or access marketplace
- Encouraged to register for full experience

2.3. Operating Environment

Platform Specifications:

- **Mobile OS:** Android 8.0+ and iOS 8.0+
- **Hardware:** Minimum 2GB RAM, 50MB storage
- **Network:** Requires active internet connection (WiFi or mobile data)

Development & Deployment:

- **Development Tool:** MIT App Inventor (visual blocks programming)
- **Backend:** Firebase (Realtime Database + Authentication + Hosting)

- **Image Storage:** Cloudinary
- **Testing:** Android emulator + physical device testing

Browser Requirements:

- Modern browsers supporting Service Workers
- **HTTPS** required for all communications

2.4. Design and Implementation Constraints

MIT App Inventor Limitations:

- Limited to visual block programming (no direct code)
- No native support for complex nested queries
- Maximum app size ~50MB after compilation
- Limited offline capability

Firebase Free Tier:

- 100 concurrent connections maximum
- 1GB total storage
- Rate limiting on reads/writes
- No built-in full-text search (*mitigation*: indexed keywords)

Image Handling:

- Cloudinary free tier limited to 10GB/month bandwidth
- Images must be optimized before upload
- *Base64 encoding* in MIT is memory-intensive

Cross-Platform Support:

- iOS support via Progressive Web App (not native)
- Native Android recommended for full feature parity
- Testing required on both platforms despite code reuse

Development Scope:

- Project prototype status; *not production-ready*
- Scalability features (caching, CDN, load balancing) excluded
- Advanced analytics and machine learning not included
- Real-time notifications may have delays

2.5 Assumptions and Dependencies

Assumptions:

- All users have access to mobile devices and have internet connectivity
- Users possess basic mobile app literacy
- University emails are available for authentication
- Firebase services remain available throughout project
- Cloudinary API availability for image uploads

Dependencies:

- **MIT App Inventor**: Platform for development and compilation
- **Firebase**: Realtime Database, Authentication, optional Cloud Functions
- **Cloudinary API**: Image upload
- **Google Account:** Required to access Firebase and MIT App Inventor
- **Internet Services**: Continuous connectivity for real-time sync

External Risks:

- Dependency on Firebase availability and pricing changes
- MIT App Inventor updates may affect compilation
- Cloudinary service disruptions

FUNCTIONAL REQUIREMENTS

4.1. Requirements Analysis and Modeling

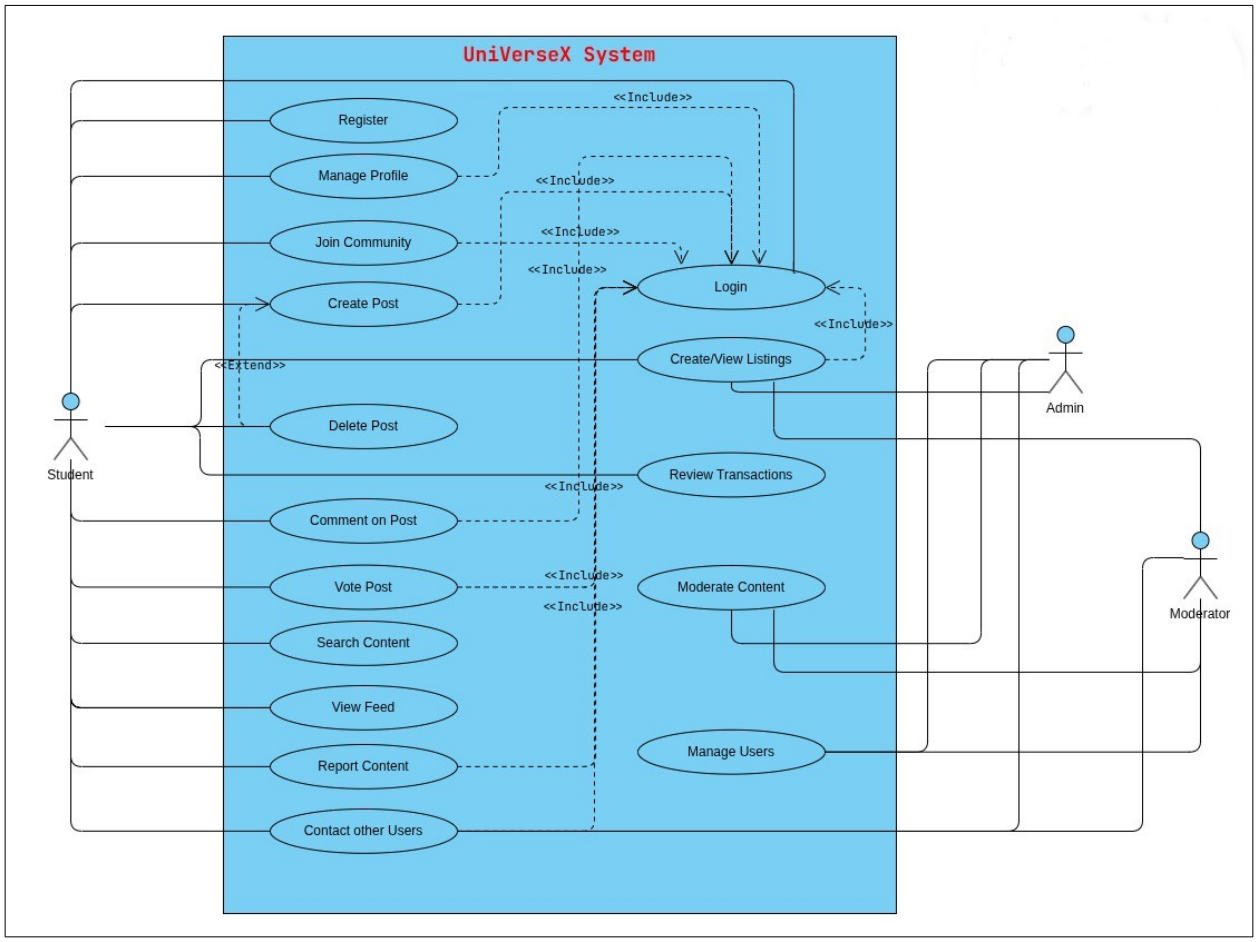
Actors:

- **User (Regular Student):** Primary actor interacting with discussion and marketplace features
- **Admin (System Administrator):** Platform-wide moderation and management
- **Moderator (Community Moderator):** Community-specific content management

Main Use Cases:

1. Register Account
2. Login to Account
3. Manage Profile
4. Create/Join Community
5. Create Post
6. Comment on Post
7. Upvote/Downvote Post
8. Search Posts/Content
9. Create Marketplace Listing
10. Contact Seller / In-App Messaging
11. Rate/Review Transaction
12. Report Content
13. Moderate Content (Remove Posts/Users)
14. View Community Feed
15. Delete Own Post

Use-Case Diagram:



4.2. Use Cases

1. Register Account

Identifier	UC - 01
Purpose	To allow users to create an account using a university email.
Actor	Student
Priority	High
Pre-conditions	■ User must not already be registered. ■ User must have valid university email.
Post-conditions	➢ User account is created successfully. ➢ Typical Course of Action.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User opens registration page	System displays form
2	User enters details	System validates input
3	User submits form	System creates account
4	-	Confirmation message is displayed

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Email already exists	System displays error message "**Account already exists**"
2	Invalid email format	System rejects input and shows validation error
3	Weak password entered	System prompts user to use a strong password

2. Login to Account

Identifier	UC - 02	
Purpose	To authenticate users and allow system access.	
Actor	Student, Moderator, Admin	
Priority	High	
Pre-conditions	User must be registered	
Post-conditions	User is logged into system	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User enters credentials	System verifies data
2	User clicks login	System authenticates
3	Success	System opens dashboard
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Wrong password	System shows error
2	Unregistered email	Access denied

3. Manage Profile

Identifier	UC - 03	
Purpose	To allow users to update personal profile information	
Actor	Student	
Priority	Medium	
Pre-conditions	User must be logged in.	
Post-conditions	Profile information is updated successfully	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User opens profile page	System displays profile data
2	User edits information	System validates changes
3	User saves changes	System updates records
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Invalid profile data entered	System rejects changes and shows error
2	Image upload fails	System displays upload failure message
3	User cancels update	System discards changes and retains old data

4. Create/Join Community

Identifier	UC - 04
Purpose	To allow users to join/create communities
Actor	Student
Priority	High
Pre-conditions	User must be logged in
Post-conditions	➢ User successfully joins an existing community **OR** ➢ User creates and becomes admin of a new community

Typical Course of Action

S#	Actor Action	System Response
1	User views communities	System shows list of available communities
2	User selects community **OR** chooses "Create Community"	System shows details OR creation form
3	User clicks "Join"	System adds user as a member
4	User enters community details (for creation) and submits	System creates new community and assigns user as admin

Alternate Course of Action

S#	Actor Action	System Response
1	User tries to join a private community	System requests approval or sends join request
2	User is already a member	System displays "**Already a member**" message
3	User submits incomplete/invalid creation form	System shows error and asks for corrections
4	Community creation fails (e.g., duplicate name)	System displays failure message and prompts retry

5. Create Post

Identifier	UC - 05
Purpose	To allow users to publish posts on profile or in communities.
Actor	Student
Priority	High
Pre-conditions	User must be logged in.
Post-conditions	Post is published.

Typical Course of Action

S#	Actor Action	System Response
1	User selects create post	System opens editor
2	User enters content	System validates
3	User submits post	System publishes

Alternate Course of Action

S#	Actor Action	System Response
1	Empty post submitted	System rejects and show validation error
2	Unsupported file format uploaded	System denies upload and shows warning
3	Network failure during submission	System does not publish post and show retry message

6. Comment on Post

Identifier	UC - 06
Purpose	Allow users to comment on posts
Actor	Student
Priority	High
Pre-conditions	■ User must be logged in. ■ Post must exist
Post-conditions	Comment is added to the post

Typical Course of Action

S#	Actor Action	System Response
1	User opens a post	System displays post details
2	User writes a comment	System accepts input
3	User submits comment	System posts comment

Alternate Course of Action

S#	Actor Action	System Response
1	Empty comment submitted	System shows error
2	Offensive language	The system detects bad language and takes action.

7. Upvote/Downvote Post

Identifier	UC - 07
Purpose	Allow users to vote on posts
Actor	Student
Priority	Low
Pre-conditions	User must be logged in.
Post-conditions	Vote is recorded

Typical Course of Action

S#	Actor Action	System Response
1	User views post	System displays post
2	User clicks upvote/downvote	System updates vote count

Alternate Course of Action

S#	Actor Action	System Response
1	User changes vote	System updates vote

8. Search Posts/Content

Identifier	UC - 08
Purpose	To allow users to search post and marketplace listings.
Actor	Student
Priority	Medium
Pre-conditions	None
Post-conditions	Search results are displayed

Typical Course of Action

S#	Actor Action	System Response
1	User enters keyword	System processes query
2	User clicks search	System searches database
3	Results found	System displays results

Alternate Course of Action

S#	Actor Action	System Response
1	No results found	System displays "**No results found**" message
2	Invalid search keyword	System ignores request and ask for valid input

9. Create Marketplace Listing

Identifier	UC - 09
Purpose	To allow users to sell or barter items
Actor	Student
Priority	Medium
Pre-conditions	User logged in
Post-conditions	Listing or transaction is processed

Typical Course of Action

S#	Actor Action	System Response
1	User opens marketplace	System shows listings
2	User clicks create listing	System shows form
3	User submits details	System publishes listing

Alternate Course of Action

S#	Actor Action	System Response
1	Invalid listing details	System rejects listing submission

10. Contact Seller / In-App Messaging

Identifier	UC - 10
Purpose	Enable communication between users
Actor	Student
Priority	Medium
Pre-conditions	User logged in
Post-conditions	Message is sent

Typical Course of Action

S#	Actor Action	System Response
1	User selects seller/simple account	System opens chat
2	User sends message	System delivers message

Alternate Course of Action

S#	Actor Action	System Response
1	Network issue	System shows error

11. Rate/Review Transaction

Identifier	UC - 11
Purpose	Allow users to rate transactions
Actor	Student
Priority	Low
Pre-conditions	Transaction completed
Post-conditions	Rating is stored

Typical Course of Action

S#	Actor Action	System Response
1	User selects transaction	System shows review option
2	User submits rating	System saves review

Alternate Course of Action

S#	Actor Action	System Response
1	Already reviewed	System shows error message

12. Report Content

Identifier	UC - 12	
Purpose	Allow users to report inappropriate content	
Actor	Student	
Priority	Medium	
Pre-conditions	User logged in	
Post-conditions	Report is submitted	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User selects report option	System shows form
2	User submits report	System records complaint
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Network issue	System shows error message

13. Moderate Content (Remove Posts/Users)

Identifier	UC - 13	
Purpose	Allow admin/moderator to manage content	
Actor	Admin/Moderator	
Priority	Medium	
Pre-conditions	Admin logged in	
Post-conditions	Content/user removed	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin views reports	System shows reported items
2	Admin removes content/user	System deletes item

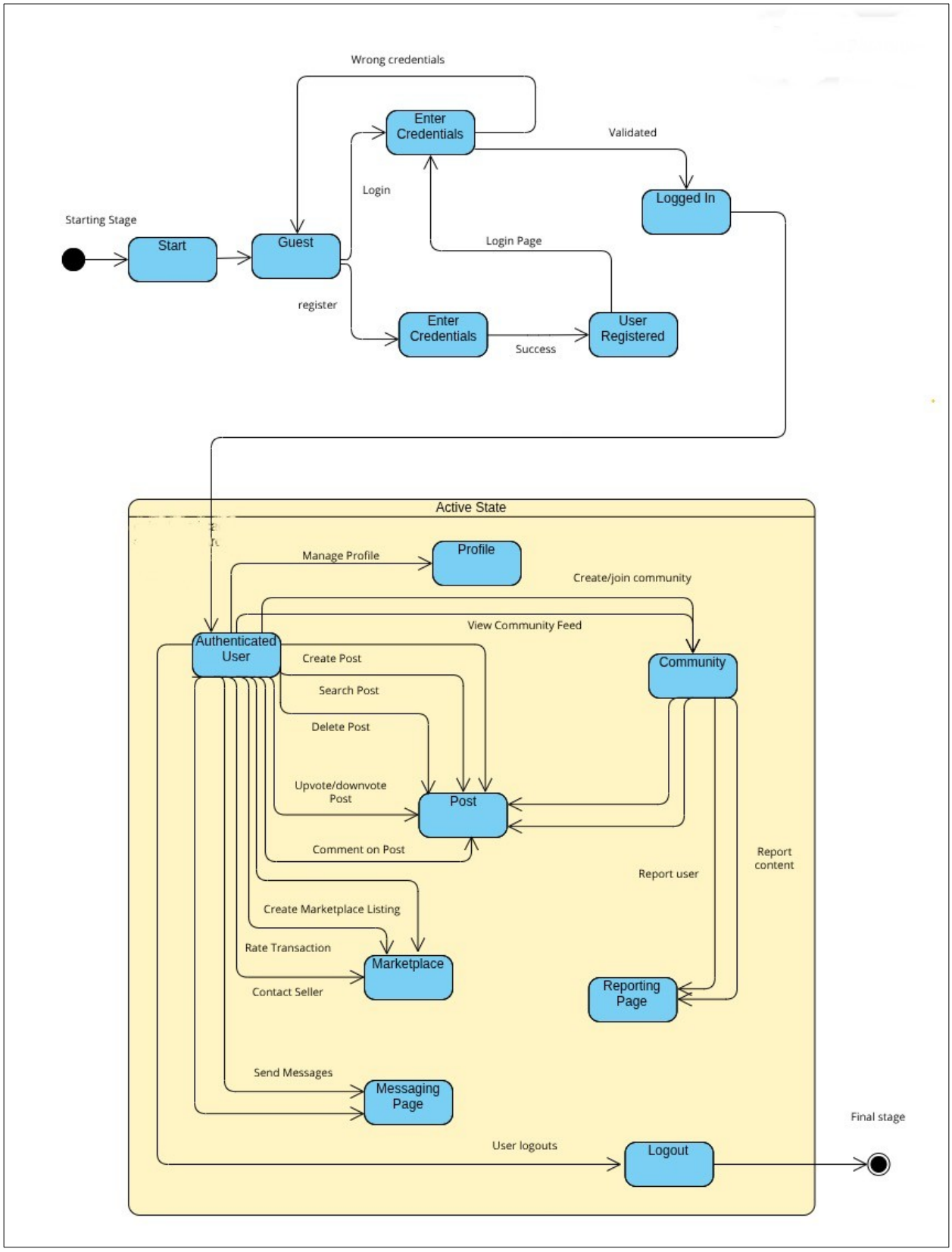
14. View Community Feed

Identifier	UC - 14	
Purpose	Allow users to view posts	
Actor	Student	
Priority	High	
Pre-conditions	Student logged in	
Post-conditions	Feed displayed	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User opens feed	System shows posts

15. Delete Own Post

Identifier	UC - 15	
Purpose	Allow users to delete their posts	
Actor	Student	
Priority	Low	
Pre-conditions	User owns the post	
Post-conditions	Post is deleted	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User selects own post	System shows options
2	User clicks delete	System removes post
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Unauthorized delete attempt	System denies action

4.3. State Diagram



State Diagram Info:

The state diagram of UniVerseX represents the different states a user or system component transitions through during interaction with the application. It illustrates how the system responds to user actions such as logging in, creating posts, interacting with content, and logging out. Initially, the system starts in an ***Unauthenticated State***, where the user can either register or log in. Upon successful authentication, the user transitions to the *Authenticated State*, gaining access to features like viewing the feed, joining communities, creating posts, commenting, and interacting with the marketplace. Each action (e.g., creating a post, sending a message, or listing an item) represents a transition that leads to corresponding states such as *Post Created*, *Comment Added*, or *Listing Published*. The system also handles alternate transitions like errors (invalid input, network failure) and moderation actions (content removal), ensuring proper flow control. Finally, the user can transition to the *Logged Out State*, returning the system to its initial state. This diagram helps visualize system behavior, ensuring all possible user interactions and system responses are clearly defined and managed.

Non-Functional Requirements

This section defines the quality attributes of the UniVerseX system. These requirements ensure that the system performs efficiently, remains secure, and provides a smooth user experience for students.

5.1. Performance Requirements

The system is designed to support real-time interaction and must respond quickly to user actions.

- The system shall load the main feed within **2–3 seconds** under normal conditions.
- The system shall reflect new posts and comments in **real-time or near real-time**.
- The system shall support multiple active users without noticeable delay.
- The system shall handle image uploads efficiently without slowing down the application.

5.2. Safety Requirements

The system must provide a safe environment for users, especially due to user-generated content and marketplace interactions.

- The system shall allow users to report inappropriate or abusive content.
- The system shall enable moderators to take action against reported content.
- The system shall prevent misuse of the platform through moderation controls.
- The system shall ensure safe interaction between buyers and sellers.

5.3. Compatibility Requirements

The system should work across supported devices and platforms.

- The system shall run on Android and iOS devices.
- The system shall function on devices with basic hardware requirements.
- The system shall require an active internet connection for operation.

5.4. Security Requirements

Security is essential to protect user data and ensure controlled access.

- The system shall allow access only to authenticated users.
- The system shall use secure methods for user authentication.
- The system shall protect user data from unauthorized access.
- The system shall ensure secure communication between user and system.

5.5. Usability Requirements

The system should be simple and easy to use for university students.

- The system shall provide a clear and user-friendly interface.
- The system shall allow users to perform key actions easily (posting, commenting, searching).
- The system shall provide feedback messages for user actions.
- The system shall maintain consistency across all screens.

5.6. Reliability Requirements

The system must remain stable and dependable during usage.

- The system shall operate with minimal downtime.
- The system shall ensure that user data is not lost during failures.
- The system shall handle errors without crashing.
- The system shall confirm successful completion of user actions.

5.7. Scalability Requirements

The system should support future growth and expansion.

- The system shall handle an increasing number of users and posts.
- The system shall support future feature enhancements.
- The system shall allow integration with more advanced services if required.